Chapter 7: Pipes

1. Overview

A pipe is a simple, declarative way to transform a value inside a template without polluting component logic with formatting code. Instead of writing `{{ formatDate(user.createdAt) }}` and maintaining a `formatDate()` helper on every component that needs it, Angular lets you write `{{ user.createdAt | date:'mediumDate' }}`. The transformation logic lives once, in a class decorated with `@Pipe`, and is reusable across the entire application.

Pipes are pure functions by design intent: given the same input, they should produce the same output, with no side effects. This purity is not just a style guideline — Angular's change detection literally treats pipes differently based on a `pure` flag, and that distinction (pure vs impure) is one of the most commonly misunderstood — and most commonly interview-tested — corners of Angular.

Pipes matter for an interview because they sit at the intersection of three topics interviewers love: template syntax, change detection, and performance. A candidate who can explain *why* an impure pipe is dangerous for large lists, or *why* `AsyncPipe` is the recommended way to consume Observables in templates, demonstrates real production experience rather than tutorial-level familiarity.

2. Core Concepts

2.1 What a Pipe Actually Is

Structurally, a pipe is a TypeScript class that implements `PipeTransform`:

```
export interface PipeTransform {
  transform(value: any, ...args: any[]): any;
}
```

It is registered with the `@Pipe` decorator, which gives it a `name` used in templates:

```
@Pipe({ name: 'myPipe' })
export class MyPipe implements PipeTransform {
  transform(value: unknown, ...args: unknown[]): unknown {
    return value;
  }
}
```

In a template, the pipe operator `|` feeds the left-hand expression as the first argument to `transform()`. Anything after a colon becomes an additional positional argument:

```
{{ amount | currency:'USD':'symbol':'1.2-2' }}
```

This compiles conceptually to:

```
currencyPipeInstance.transform(amount, 'USD', 'symbol', '1.2-2');
```

2.2 Built-in Pipes

Angular ships a set of commonly needed pipes in `@angular/common`:

Pipe	Purpose	Example
`DatePipe`	Formats a date according to locale/format string	`{{ today | date:'yyyy-MM-dd' }}`
`UpperCasePipe`	Transforms string to uppercase	`{{ name | uppercase }}`
`LowerCasePipe`	Transforms string to lowercase	`{{ name | lowercase }}`
`TitleCasePipe`	Capitalizes the first letter of each word	`{{ title | titlecase }}`
`CurrencyPipe`	Formats a number as currency	`{{ price | currency:'EUR' }}`
`DecimalPipe` (`number`)	Formats a number with digit info	`{{ pi | number:'1.0-2' }}`
`PercentPipe`	Formats a number as a percentage	`{{ 0.256 | percent }}`
`JsonPipe`	Serializes a value via `JSON.stringify` (great for debugging)	`{{ obj | json }}`
`SlicePipe`	Returns a slice of an array or string	`{{ items | slice:0:5 }}`
`AsyncPipe`	Subscribes to an `Observable`/`Promise` and unwraps emitted values	`{{ data$ | async }}`
`KeyValuePipe`	Turns an object or `Map` into an iterable of `{key, value}` pairs	`*ngFor="let e of obj | keyvalue"`
`I18nPluralPipe` / `I18nSelectPipe`	Locale-aware pluralization/selection	rarely used directly, powers ICU expressions

`DatePipe`, `CurrencyPipe`, `DecimalPipe`, and `PercentPipe` are all **locale-aware**: they use `LOCALE_ID` (injected token) to decide formatting conventions (decimal separators, currency symbols, date order). Changing the app's locale changes their output without any code change.

`SlicePipe` and `KeyValuePipe` deserve a note: `SlicePipe` is technically impure in behavior when applied to arrays that mutate in place (though registered as pure — see gotchas below), and `KeyValuePipe` re-sorts keys by default (alphabetically by key, unless you supply a `compareFn`), which surprises people who expect object insertion order to be preserved.

2.3 Pure vs Impure Pipes — The Central Distinction

Every pipe declares purity via the `pure` property on `@Pipe`:

```
@Pipe({ name: 'example', pure: true }) // default is true
```

Pure pipe:

- Angular caches the pipe's last input value(s) and last output.
- On each change detection cycle, Angular compares the new input reference to the cached one using a form of identity check (`===`, roughly — technically Angular's internal `looseIdentical` for primitives, and reference equality for objects/arrays).
- If the input reference (and any parameters) is unchanged, Angular **skips calling `transform()` entirely** and reuses the memoized output.
- `transform()` only re-executes when the reference changes — meaning if you mutate an array or object in place (`arr.push(x)`) rather than creating a new reference (`arr = [...arr, x]`), a pure pipe will NOT pick up the change on the next CD cycle, because the reference is identical.

Impure pipe:

- Declared with `pure: false`.
- Angular calls `transform()` on **every single change detection cycle**, regardless of whether the input reference changed — even if nothing in the whole application actually changed value-wise.
- This is expensive: if change detection runs on a keystroke, a mouse move (with certain zone patches), an HTTP response, a timer tick, anything — the impure pipe's `transform()` runs again, synchronously, in that cycle.
- Impure pipes exist specifically so a pipe *can* react to in-place mutation of arrays/objects, or to external/mutable state (like a clock), that a pure pipe would miss.

The built-in `AsyncPipe` and `JsonPipe`... actually `JsonPipe` is pure by default in modern Angular but conceptually behaves observably like it re-stringifies on each check if given a new reference; the pipe that is genuinely impure in the framework is `AsyncPipe` — it is marked `pure: false` because its entire job is to track a mutable, evolving stream over time; the reference to the Observable itself never changes, yet new values keep arriving, so a pure-pipe cache-by-reference model would never re-render.

Why this matters for performance: an impure pipe placed inside a large `*ngFor`, or one doing expensive work (deep object comparison, complex string parsing, filtering a large array) will silently become a hot path executed dozens of times per second under normal user interaction, because Angular's default (non-`OnPush`) change detection runs on virtually every asynchronous browser event via Zone.js. This is the single most common “why is my app janky” pipe-related interview trap.

2.4 The AsyncPipe In Depth

`AsyncPipe` is arguably the most important pipe to understand deeply because it's a subscription-management pattern disguised as a pipe.

What it does, mechanically:

1. `ngOnInit`/first CD pass — when the template first evaluates `data$ | async`, `AsyncPipe.transform(obs)` checks if it already has a subscription to this exact object. If not, it calls `.subscribe()` on the Observable (or `.then()` on the Promise) and stores the subscription.
2. On each emission, the pipe stores the latest emitted value internally and calls

`ChangeDetectorRef.markForCheck()` to schedule a check of the component (this is essential — otherwise, with `OnPush` components, the newly-arrived value would never be rendered, since nothing else would trigger CD).

3. On every subsequent CD cycle, `transform()` (impure, so it's called each cycle) simply returns the last cached emitted value — it does NOT re-subscribe.
4. If the bound expression's *reference* changes (i.e., you assign `this.data$ = newObservable$`), `AsyncPipe` detects the new Observable instance, unsubscribes from the old one, and subscribes to the new one.
5. **On component/directive destruction (`ngOnDestroy`), `AsyncPipe` automatically unsubscribes** from whatever Observable it's currently attached to. This is its single greatest practical value: it eliminates an entire class of memory leaks that plague manual `.subscribe()` calls in `ngOnInit` without a matching teardown in `ngOnDestroy`.

Because of this lifecycle-bound automatic unsubscription, `| async` is the idiomatic, recommended way to render Observable data in templates, and is a cornerstone of push-based, `OnPush`-friendly Angular architectures (subscribe never in the component class, only in the template).

A subtlety: if the *same* Observable is referenced by `| async` in multiple places in the same template, older Angular versions could create multiple subscriptions (one per pipe instance) — mitigated in practice by using the `as` template syntax to alias the unwrapped value once:

```
<ng-container *ngIf="data$ | async as data">
  <p>{{ data.name }}</p>
  <p>{{ data.email }}</p>
</ng-container>
```

This subscribes once and reuses `data` for both bindings.

2.5 Pipe Parameters and Chaining

Pipe parameters are separated by colons and are just additional arguments to `transform`:

```
{{ value | slice:1:4 }}
{{ date | date:'short':'UTC':'en-US' }}
```

Pipes can be **chained** — the output of one becomes the input of the next, read left to right:

```
{{ title | lowercase | titlecase }}
{{ user.birthDate | date:'longDate' | uppercase }}
```

Order matters and each pipe in the chain is evaluated independently by change detection (a pure pipe earlier in the chain can still short-circuit its own recomputation even if a later pipe in the chain is impure, and vice versa — each pipe instance has its own memoization state).

2.6 Custom Pipes

Creating a custom pipe requires: implementing `PipeTransform`, decorating with `@Pipe`, and declaring/importing it (in an `NgModule`'s `declarations`, or directly in a standalone component's `imports` array if the pipe itself is `standalone: true`).

Custom pipes are the natural home for: text truncation, currency conversion with a live exchange rate, filtering/sorting arrays (with heavy caveats — see gotchas), permission-based visibility transforms, safe-HTML sanitization wrappers, and unit conversions.

3. Code Examples

3.1 A Basic Pure Custom Pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate',
  standalone: true, // Angular 14+: usable without an NgModule
})
export class TruncatePipe implements PipeTransform {
  transform(value: string | null | undefined, limit = 50, suffix = '…'): string {
    if (!value) {
      return '';
    }
    return value.length > limit ? value.slice(0, limit).trimEnd() + suffix : value;
  }
}
```

Usage:

```
<p>{{ article.body | truncate:120:'... read more' }}</p>
```

3.2 An Impure Custom Pipe (Filtering an Array)

This is the classic "search box filtering a list" pipe. Note it must be impure because the array reference (`items`) typically does not change when the *filter term* changes — only a sibling input changes.

```
import { Pipe, PipeTransform } from '@angular/core';

interface Product {
  name: string;
  category: string;
}

@Pipe({
  name: 'filterByName',
  pure: false, // required: 'term' is a separate argument, not part of 'items' reference
  standalone: true,
})
```

```
export class FilterByNamePipe implements PipeTransform {
  transform(items: Product[] | null, term: string): Product[] {
    if (!items || !term) {
      return items ?? [];
    }
    const lowerTerm = term.toLowerCase();
    return items.filter(item => item.name.toLowerCase().includes(lowerTerm));
  }
}
```

```
<input [(ngModel)]="searchTerm" placeholder="Search products" />
<ul>
  <li *ngFor="let p of products | filterByName:searchTerm">{{ p.name }}</li>
</ul>
```

Production caveat: this pattern is explicitly discouraged by the Angular team for large or frequently-changing lists, precisely because it re-filters on every CD cycle. The preferred alternative is to perform filtering in the component (e.g., via a `computed()` signal, or updating a `filteredProducts` array reactively when `searchTerm` changes), keeping the template pipe-free or using a pure pipe fed by an already-filtered, reference-stable array.

3.3 A Pipe With Dependency Injection (Locale/Service-Aware)

Pipes are injectable classes, so they can use Angular's DI just like services:

```
import { Pipe, PipeTransform, inject } from '@angular/core';
import { ExchangeRateService } from './exchange-rate.service';

@Pipe({
  name: 'convertCurrency',
  standalone: true,
})
export class ConvertCurrencyPipe implements PipeTransform {
  private exchangeRateService = inject(ExchangeRateService);

  transform(amountUsd: number, targetCurrency: string): number {
    const rate = this.exchangeRateService.getRateSync(targetCurrency);
    return Math.round(amountUsd * rate * 100) / 100;
  }
}
```

3.4 Custom Async-Style Pipe Using ChangeDetectorRef (Mimicking AsyncPipe)

This demonstrates *why* `AsyncPipe` needs `markForCheck()` — building a miniature version clarifies the internal mechanism:

```
import { OnDestroy, Pipe, PipeTransform, ChangeDetectorRef, inject } from '@angular/core';
import { Observable, Subscription } from 'rxjs';
```

```
@Pipe({
  name: 'myAsync',
  pure: false,
  standalone: true,
})
export class MyAsyncPipe implements PipeTransform, OnDestroy {
  private cdr = inject(ChangeDetectorRef);
  private subscription: Subscription | null = null;
  private latestValue: unknown = null;
  private source$: Observable<unknown> | null = null;

  transform<T>(source: Observable<T> | null): T | null {
    if (source !== this.source$) {
      this.dispose();
      this.source$ = source;
      if (source) {
        this.subscription = source.subscribe(value => {
          this.latestValue = value;
          this.cdr.markForCheck(); // essential for OnPush components
        });
      }
    }
    return this.latestValue as T | null;
  }

  ngOnDestroy(): void {
    this.dispose();
  }

  private dispose(): void {
    this.subscription?.unsubscribe();
    this.subscription = null;
    this.latestValue = null;
  }
}
```

3.5 Registering a Pipe (NgModule vs Standalone)

```
// NgModule-based (legacy pattern)
@NgModule({
  declarations: [TruncatePipe],
  exports: [TruncatePipe],
})
export class SharedPipesModule {}
```

```ts
// Standalone (modern pattern, Angular 14+)
@Component({
  selector: 'app-article-card',
  standalone: true,
  imports: [TruncatePipe, CommonModule],
  template: `<p>{{ body | truncate:100 }}</p>`,
})
export class ArticleCardComponent {}
```

3.6 KeyValuePipe With a Custom Comparator

```ts
@Component({
  standalone: true,
  imports: [KeyValuePipe],
  template: `
    <div *ngFor="let entry of scores | keyvalue:byValueDesc">
      {{ entry.key }}: {{ entry.value }}
    </div>
  `,
})
export class ScoreboardComponent {
  scores: Record<string, number> = { alice: 42, bob: 91, carol: 17 };

  byValueDesc = (a: KeyValue<string, number>, b: KeyValue<string, number>): number =>
    b.value - a.value;
}
```

4. Internal Working

4.1 Compile-Time Wiring

When the Angular Ivy template compiler (`ngtsc`) processes a template containing `{{ value | myPipe:arg }}`, it does not generate a plain function call. It generates specific instructions in the component's generated render function:

- `ɵɵpipe(index, 'myPipe')` — registers a slot in the component's `LView` for an instance of the pipe, resolved from the pipe's injectable factory. This happens once, during the **creation pass** of that view.
- `ɵɵpipeBind1(index, offset, value)` (or `pipeBind2`, `pipeBind3`, `pipeBindV` for more arguments) — invoked during the **update pass**, on every change detection run for that view. This is the actual call site of `transform()`.

Each pipe usage in a template gets its own dedicated pipe instance stored in the `LView` (the internal data structure backing that template's view), not a shared singleton across the app — so two different `{{ x | myPipe }}` expressions in two different components each get their own `MyPipe` instance, each with independent memoized state.

4.2 The `pipeBind*` Memoization Check

Inside `pipeBind1` (simplified conceptually), Angular does roughly:

```
function pipeBind1(lView, pipeIndex, offset, value) {
  const pipeInstance = lView[pipeIndex];
  return unwrapValue(lView, isPure(pipeInstance)
    ? pureFunction1(lView, offset, pipeInstance.transform, value)
    : pipeInstance.transform(value));
}
```

`pureFunction1` (and its siblings for more args) is the memoization layer: it compares the new `value` (and any additional args) against values cached in a reserved slot of the `LView` from the previous invocation, using identity comparison. If unchanged, it returns the previously cached transform result without calling `transform()` at all. If changed, it calls `transform()`, caches the new inputs and output, and returns the fresh result.

For an impure pipe, this memoization check is bypassed entirely — the generated code calls `pipeInstance.transform(...)` directly and unconditionally on every update pass, i.e., every time that view is checked during change detection.

4.3 Why This Makes Pure Pipes "Cheap"

"Cheap" here specifically means: cheap *relative to leaving the transformation as a method call in the template* (e.g., `{{ formatValue(x) }}`), because method calls in templates have **no memoization at all** — they re-execute on every CD cycle unconditionally, exactly like an impure pipe would, but without even the *option* of being pure. A pure pipe gives you the opportunity to skip recomputation when inputs are referentially stable; a template method call never does. This is why "avoid calling methods in templates, use pure pipes instead" is standard Angular performance guidance — the underlying reason is this exact memoization mechanism.

4.4 Interaction With Change Detection Strategy

Pipes operate at the level of *view update*, which itself only happens if the view is checked at all:

- With `ChangeDetectionStrategy.Default`, Angular checks (and therefore re-evaluates) every component's bindings — including pipe update calls — on every CD cycle, triggered by Zone.js patches around async APIs (`setTimeout`, DOM events, XHR/fetch completions, Promise resolution, etc.).
- With `ChangeDetectionStrategy.OnPush`, a component's view is only checked when: an `@Input()` reference changes, an event originates from within the component's own template, an Observable bound via `| async` emits (because `AsyncPipe` calls `markForCheck()`), or `markForCheck()` / `detectChanges()` is called manually (e.g. from a signal update, or `ChangeDetectorRef`).
- Regardless of strategy, once a view *is* being checked, pure pipes still apply their memoization (skipping `transform()` if inputs are unchanged), and impure pipes still run every time that view is checked.

This is precisely why impure pipes and `OnPush` are commonly discussed together: `OnPush` reduces *how often* a component's views are checked; pipe purity determines, *within* each check, whether the pipe actually recomputes. Both levers exist because Angular's change detection is fundamentally about deciding what to re-examine, not about tracking fine-grained dependencies like signals or reactive frameworks do.

4.5 Angular Signals and the Future of This Model

With the introduction of Signals, Angular is moving toward fine-grained reactivity where a `computed()` only re-evaluates when its actual signal dependencies change — conceptually similar to a "perfectly pure pipe" but tracked automatically rather than via reference-equality on explicit arguments. Pipes still exist and are fully supported (and Angular provides `toSignal()` to bridge Observables into signals, an alternative to `| async` in signal-based components), but understanding the pipe memoization model remains foundational to understanding why Angular change detection behaves the way it does historically and in mixed codebases.

5. Edge Cases & Gotchas

- **Mutating arrays/objects bypasses pure pipes.** `this.items.push(x)` does not change the `items` reference, so a pure pipe filtering/sorting `items` will not re-run. Fix: reassign with a new reference (`this.items = [...this.items, x]`) or mark the pipe impure (with a performance cost).
- **Impure pipes inside `*ngFor` are a classic performance foot-gun.** A pipe like `{{ item | expensiveTransform }}` inside a loop of 500 rows, if impure, executes 500 times on *every* change detection cycle — including ones triggered by unrelated UI events like scrolling or hovering (if those trigger zone-patched events).
- **Never put filtering/sorting pipes directly in templates for large datasets** without a strong reason. Angular's official style guide recommends doing filtering in the component/service layer and exposing an already-filtered array (or better, a signal/computed), not via an impure pipe re-executed every cycle.
- `DatePipe` / `CurrencyPipe` / `DecimalPipe` **require** `registerLocaleData()` for any locale other than the default (`en-US`) baked into Angular — forgetting this throws a runtime error (`Missing locale data for the locale "xx"`) even though everything compiles fine.
- `AsyncPipe` **and multiple subscriptions.** Using `data$ | async` multiple times in one template historically could create redundant subscriptions to the same Observable if it's not shared/cold. Always prefer the `as` alias pattern, or ensure the Observable is `shareReplay`'d if used in several places, especially if it triggers an HTTP call (cold Observable = new HTTP request per subscription!).
- `AsyncPipe` **combined with** `OnPush` **and manual** `markForCheck` **conflicts.** If you also mutate `ChangeDetectorRef.detach()` on a component using `| async`, the pipe's internal `markForCheck()` calls become meaningless because the view is detached — a common source of "why isn't my async data showing up" bugs.
- **Pipes cannot be used in TypeScript code directly** (only in templates) unless you manually instantiate the pipe class (`new DatePipe(locale).transform(...)`) or inject it — a pipe is not automatically available as a plain function in `.ts` files.
- `JsonPipe` **and circular references.** Since it wraps `JSON.stringify`, passing an object with circular references throws at runtime — easy to hit when accidentally binding a DOM node, an Angular component instance, or a circular parent/child data model.
- `SlicePipe` **is registered** `pure: true`**, yet operates on arrays**, meaning slicing a mutated-in-place array won't update — same trap as any pure pipe over a mutable reference.
- **Null/undefined handling is the pipe author's responsibility.** Angular does not automatically guard `transform()` against `null` / `undefined` input; a custom pipe that calls `.toLowerCase()` on a possibly-null string will throw. Defensive guards (`value ?? ''`) are essential, especially since template expressions frequently pipe optional/async data (`user?.name | uppercase`).

- **Chaining order changes both output and performance.** `{{ list | filterByName:term | slice:0:10 }}` filters the *entire* list on every keystroke, then slices; reversing the order (`slice` first) would produce wrong results (slicing before filtering loses matches outside the initial slice window) — a subtle correctness bug, not just a performance one.
- **Standalone pipes must be explicitly imported** into every standalone component that uses them (in the `imports` array) — forgetting this produces a compile-time/template error like `NG0304: 'myPipe' pipe not found`, which is a very common mistake when migrating an NgModule app to standalone.
- **Pipe name collisions across libraries.** Because pipe resolution in templates is name-based (a string), two differently-implemented pipes named the same (e.g., two `search` pipes from different shared libraries) will silently shadow one another depending on import order — there is no compiler error for "duplicate pipe name" the way there is for duplicate component selectors in some tooling.

6. Interview Questions & Answers

Q1. What is a pipe in Angular, and what's the syntax to apply one in a template?
A pipe is a class implementing `PipeTransform` that transforms a bound value for display, without changing the underlying data. Syntax uses the `|` character: `{{ expression | pipeName:arg1:arg2 }}`. The expression's value becomes `transform()`'s first parameter; each colon-separated token becomes a subsequent parameter.

Q2. Name five built-in Angular pipes and what each does.
`DatePipe` (date formatting), `CurrencyPipe` (currency formatting), `UpperCasePipe` / `LowerCasePipe` (case transforms), `DecimalPipe` / `number` (numeric formatting), `JsonPipe` (stringifies for debugging), `SlicePipe` (array/string slicing), `AsyncPipe` (unwraps Observables/Promises), `KeyValuePipe` (iterates object entries), `PercentPipe` (percentage formatting).

Q3. What is the difference between a pure and an impure pipe?
Interviewer intent: This checks whether the candidate understands Angular's change detection memoization model, not just pipe syntax.
A pure pipe (the default, `pure: true`) is memoized by Angular: its `transform()` is only re-invoked when the input reference (or any argument) changes between change detection cycles, per a reference/identity comparison. An impure pipe (`pure: false`) has its `transform()` called on every single change detection cycle regardless of whether anything actually changed, making it far more expensive. Impure pipes are needed when you must react to in-place mutation of an object/array or to time-based/external state that doesn't manifest as a new reference (this is exactly why `AsyncPipe` is impure — the Observable reference is stable, but new values keep arriving).

Q4. Why does a pure pipe fail to update when you push a new item into an array bound to it?
Because pure pipe memoization compares the new input's *reference* to the previously cached one. `array.push(x)` mutates the array in place — the reference is identical — so Angular's `pipeBind` memoization concludes "nothing changed" and skips calling `transform()` again. The fix is to always produce a new reference on update (`this.array = [...this.array, x]`), or to mark the pipe impure (accepting the performance trade-off).

Q5. Why is `AsyncPipe` marked as impure?
Because its entire purpose is to unwrap values from a mutable, time-varying source (an Observable or Promise) whose *reference* does not change even as new values are emitted. If it were pure, Angular's memoization would cache the first emitted value forever (since the Observable reference never changes) and never show subsequent emissions. Being impure means `transform()` runs on every CD cycle, but internally it

just returns the last cached emitted value cheaply (it does not re-subscribe) — the actual work of listening for new data happens once, via the subscription set up on first use, and updates are pushed via `markForCheck()`, not by polling in `transform()`.

Q6. What are the two responsibilities `AsyncPipe` handles automatically that manual subscription in a component does not?
Interviewer intent: Probing for practical RxJS-in-Angular hygiene — the classic memory-leak question in disguise.
(1) Subscription lifecycle: it subscribes on first binding and unsubscribes automatically in its own `ngOnDestroy` when the host view is destroyed, eliminating manual `ngOnDestroy` teardown code and the memory leaks caused by forgetting it. (2) Change detection integration: after every emission, it calls `ChangeDetectorRef.markForCheck()`, ensuring the new value is rendered even in `OnPush` components, which a naive `this.value = x` assignment inside a manual `.subscribe()` callback would not otherwise guarantee under `OnPush`.

Q7. How would you write a custom pipe that formats a number of bytes into a human-readable string like "1.5 MB"?

```
@Pipe({ name: 'fileSize', standalone: true })
export class FileSizePipe implements PipeTransform {
  private units = ['B', 'KB', 'MB', 'GB', 'TB'];
  transform(bytes: number | null | undefined, precision = 1): string {
    if (bytes == null || isNaN(bytes)) return '';
    if (bytes === 0) return '0 B';
    const exponent = Math.min(Math.floor(Math.log(bytes) / Math.log(1024)),
this.units.length - 1);
    const value = bytes / Math.pow(1024, exponent);
    return `${value.toFixed(precision)} ${this.units[exponent]}`;
  }
}
```

This is naturally pure — same byte count always yields the same string — so no `pure: false` is needed.

Q8. Can pipes be chained, and does order matter?
Yes: `{{ value | pipeA | pipeB }}` applies `pipeA` first, then feeds its output into `pipeB`. Order matters both semantically (e.g., filtering before slicing gives different results than slicing before filtering) and for change detection (each pipe in the chain maintains its own independent memoization slot, so an upstream pure pipe can still skip work even if a downstream pipe in the same chain is impure, and vice versa).

Q9. What happens internally when the Ivy compiler encounters `{{ value | myPipe:arg }}` in a template?
Interviewer intent: Distinguishes candidates who've only used pipes from those who understand Ivy's instruction-based compilation model.
During the creation pass of the component's view, the compiler emits a `ɵɵpipe()` instruction that resolves and stores an instance of the pipe (via its factory/DI) into a reserved slot of the component's `LView`. During the update pass — run every change detection cycle for that view — it emits a `ɵɵpipeBind1()` (or `pipeBind2/3/V` depending on arg count) call. For a pure pipe, `pipeBind*` first checks a memoized cache of previous inputs (via `pureFunction*`); if the input(s) match by reference, it returns the cached output without calling `transform()`. If they differ, or if the pipe is impure (bypassing the cache entirely), it invokes `pipeInstance.transform(...)` and, for pure pipes, updates the cache.

Q10. Why is `{{ formatValue(x) }}` (a component method call in a template) generally worse for performance than `{{ x | formatValuePipe }}`?
A method call in a template has no built-in memoization — Angular calls it unconditionally on every check of that view, every CD cycle, regardless of whether `x` changed. A pure pipe, by contrast, is memoized by input reference, so if `x` hasn't changed since the last cycle, the transformation is skipped entirely. For expensive transformations invoked frequently (e.g., inside large lists, or components checked very often), this difference compounds significantly.

Q11. If you need to filter or sort a list for display, why is doing it via an impure pipe generally discouraged, and what's the recommended alternative?
An impure filter/sort pipe recomputes on every change detection cycle of the view it's in — which, without `OnPush`, is every zone-triggered event across the whole app, not just events relevant to the list. For a large array or an expensive comparator, this becomes a serious, silent performance drain, especially since it's invisible in code review (it just looks like a template pipe). The recommended alternative is to perform the filtering/sorting in the component or a service — updating a `filteredItems` array (or a `computed()` signal) only when the actual filter criteria change — and bind the template to that already-processed, reference-stable result, optionally with a pure pipe on top for any remaining lightweight, reference-safe transform.

Q12. How do you register a pipe for use with a standalone component versus an NgModule-based component?
For a standalone pipe (`standalone: true` on the `@Pipe` decorator, the default for pipes generated in modern Angular CLI versions), each standalone component that uses it must list it in its own `imports: []` array. For an NgModule-based setup, the pipe is added to the module's `declarations` (and `exports`, if other modules need it), and any component using it must belong to a module that either declares or imports that pipe's module. Mixing conventions (a non-standalone pipe declared only in one module, used from an unrelated module) produces the `NG0304: pipe not found` template error.

Q13. What's the danger of using `JsonPipe` on an object that contains a reference back to a component instance or DOM node?
`JsonPipe` calls `JSON.stringify()` internally. If the object graph has a circular reference (common when accidentally binding something like a parent component reference, a DOM element, or certain framework objects with back-pointers), `JSON.stringify` throws a `TypeError: Converting circular structure to JSON` at runtime, crashing that render. It's best used for plain data/debug objects, not framework or DOM-adjacent objects.

Q14. How does `ChangeDetectionStrategy.OnPush` interact with pipe purity?
Interviewer intent: Tests whether the candidate can separate "how often a view is checked" (CD strategy) from "whether a pipe recomputes during that check" (pipe purity) — two different axes people often conflate. They are complementary but distinct mechanisms. `OnPush` controls *whether a component's view is checked at all* during a given CD pass (only checked on new `@Input` references, internal DOM events, `AsyncPipe` emissions calling `markForCheck()`, or manual `markForCheck()`/signal updates). Pipe purity controls, *given that the view is being checked*, whether a specific pipe's `transform()` actually re-executes (pure: only if inputs changed by reference; impure: always). An impure pipe inside an `OnPush` component is checked less often overall than the same pipe in a `Default`-strategy component, but every time it *is* checked, the impure pipe still unconditionally recomputes — `OnPush` reduces frequency of checks, not the pipe's per-check cost.

7. Quick Revision Cheat Sheet

- **Pipe = class implementing `PipeTransform.transform(value, ...args)`**, applied in templates with

`|`.

- **Pure (default, `pure: true`)**: memoized by input reference; `transform()` skipped if inputs unchanged since last CD cycle.
- **Impure (`pure: false`)**: `transform()` runs on *every* CD cycle, no matter what — expensive, use sparingly.
- **Mutating an array/object in place does not trigger a pure pipe to re-run** — always create a new reference, or go impure.
- `AsyncPipe` **is impure by necessity** (stable Observable reference, changing emitted values); it auto-subscribes, auto-unsubscribes on destroy, and calls `markForCheck()` on each emission — solves both memory leaks and `OnPush` staleness.
- Prefer `data$ | async as data` to avoid duplicate subscriptions when used multiple times in one template.
- Built-ins: `date`, `uppercase` / `lowercase` / `titlecase`, `currency`, `number` / `decimal`, `percent`, `json`, `slice`, `keyvalue`, `async`.
- `DatePipe` / `CurrencyPipe` / `DecimalPipe` / `PercentPipe` are locale-aware via `LOCALE_ID`; non-default locales need `registerLocaleData()`.
- **Chaining**: `{{ x | a | b }}` — left to right; order affects both correctness and cost.
- **Compiler internals**: `ɵɵpipe()` (creation, instantiates pipe into `LView` slot) + `ɵɵpipeBindN()` (update pass, per CD cycle) — pure pipes route through `pureFunctionN` memoization; impure pipes call `transform()` directly, unconditionally.
- Avoid method calls in templates for the same reason you avoid impure pipes: no memoization, recomputed every CD cycle.
- Avoid filter/sort pipes on large/frequently-changing lists — do it in the component/service layer instead, or precompute with signals.
- Standalone pipes need explicit `imports: []` in every consuming standalone component; NgModule pipes need `declarations` / `exports`.
- Guard against `null` / `undefined` inside custom `transform()` methods — Angular does not do this for you.
- `OnPush` changes *how often* a view is checked; pipe purity changes *whether transform() runs given a check* — independent, complementary levers.

Created By - Durgesh Singh